

Capítulo 1: Configuración y Tooling Esencial

Como arquitecto de software senior y redactor técnico, este capítulo te proporcionará la base operativa indispensable para comenzar tu viaje en Scala, cubriendo el entorno de ejecución, las herramientas de construcción modernas y las mejores prácticas para configurar tu espacio de trabajo.

1. Explicación Teórica: El Ecosistema JVM

Scala, acrónimo de *Scalable Language* (Lenguaje Escalable), fue creado por Martin Odersky en la École Polytechnique Fédérale de Lausanne (EPFL) de Suiza. Su propósito fundamental fue **fusionar la Programación Orientada a Objetos (POO) y la Programación Funcional (PF) en una ontología singular**.

La JVM como Plataforma Base

El diseño de Scala está intrínsecamente ligado a la **Máquina Virtual de Java (JVM)**.

- **Compilación y Ejecución:** El compilador de Scala (`scalac`) toma el código fuente (`.scala`) y lo convierte en **Java Bytecode** (`.class`), que es el formato que la JVM puede ejecutar.
- **Interoperabilidad:** Correr en la JVM le otorga a Scala una ventaja estratégica: **acceso completo y sin fisuras a todo el vasto ecosistema de librerías de Java**.
- **Requisito Básico:** Para desarrollar en Scala, es imprescindible tener instalado el **Java Development Kit (JDK)**.

Tooling Moderno (El Estándar 2025)

El ecosistema de herramientas de Scala ha evolucionado para simplificar la experiencia, especialmente con la llegada de Scala 3. El enfoque ha pasado de depender exclusivamente del *Scala Build Tool* (`sbt`) a herramientas de *scripting* ligeras:

- **Coursier (`cs`):** Es la herramienta recomendada para la instalación. Se utiliza para gestionar y asegurar que la JVM, el compilador (`scalac`) y las herramientas de construcción estén configurados correctamente.
- **Scala CLI:** Es la herramienta moderna y por defecto para ejecutar scripts, prototipos y microservicios que residen en un solo archivo. Reemplaza la antigua necesidad de un archivo de configuración complejo para casos simples.
- **sbt (Scala Build Tool):** Sigue siendo la herramienta de elección para proyectos grandes, con estructuras modulares complejas y gestión avanzada de dependencias.

2. Instalación y Entorno (El Estándar 2025)

Olvida las instalaciones manuales de ZIPs. El estándar industrial hoy es **Coursier** (`cs`). Es un gestor de artefactos que instala Java, Scala y las herramientas de construcción automáticamente.

A. Instalación Rápida (Mac/Linux)

Abre tu terminal y ejecuta:

```
1  # 1. Instalar Coursier (El instalador oficial)
2  curl -fL https://github.com/coursier/launchers/raw/master/cs-x86_64-pc-
   linux.gz | gzip -d > cs && chmod +x cs && ./cs setup
3
4  # 2. Verificar la instalación (Reinicia la terminal antes)
5  cs java --version # Debería mostrar OpenJDK 17 o 21
6  scalac --version  # Debería mostrar Scala 3.x.x
7  sbt --version      # Debería mostrar 1.9.x o superior
```

(En Windows, usa el instalador `.exe` desde la web oficial de Coursier o WSL2).

B. Scala CLI vs. SBT: ¿Cuál usar?

- **Scala CLI:** Para scripts, prototipos rápidos y archivos sueltos. (Ej: `scala-cli run Main.scala`).
- **SBT (Scala Build Tool):** Para **proyectos de ingeniería**. Es lo que usarás en empresas para construir backends, librerías o jobs de Spark.

3. Anatomía de un Proyecto SBT

A diferencia de Python o JS, Scala requiere una estructura de directorios estricta (convención Maven).

Árbol de Directorios Estándar

Plaintext

```
1  mi-proyecto-scala/
2  |— build.sbt          <-- El cerebro del proyecto (dependencias y
   configuración)
3  |— project/
4  |   |— build.properties <-- Define la versión de sbt a usar
5  |— src/
6  |   |— main/
```

```

7  |   |   | resources/      <-- Configuración (application.conf,
   |   |   | logback.xml)
8  |   |   |   | scala/      <-- TU CÓDIGO FUENTE AQUÍ
9  |   |   |   |   | Main.scala
10 |   |   |   | test/
11 |   |   |   |   | scala/    <-- Tus tests unitarios (ScalaTest / Munit)
12 |   |   |   | target/      <-- Artefactos compilados (ignorar en git)

```

Tu Primer `build.sbt` (Scala 3)

Este es el archivo mínimo viable para un proyecto moderno.

```

1  val scala3Version = "3.3.1" // Usa siempre una versión LTS
2
3  lazy val root = project
4    .in(file("."))
5    .settings(
6      name := "mi-primer-proyecto",
7      version := "0.1.0-SNAPSHOT",
8
9      scalaVersion := scala3Version,
10
11     // Dependencias externas (Formato: "GrupoID" %% "Artefacto" %
12     "Versión")
13     libraryDependencies += Seq(
14       "org.scalameta" %% "munit" % "0.7.29" % Test, // Framework de
15       testing ligero
16       "com.lihaoyi"   %% "os-lib" % "0.9.1"          // Librería útil para
17       sistema de archivos
18     )
19   )

```

2. Sintaxis, Métodos de Ejecución y Consola Interactiva

La forma en que se escribe y se ejecuta el código refleja la filosofía de Scala, promoviendo la inmutabilidad y la concisión.

A. La Consola Interactiva: REPL

El **REPL** (*Read, Evaluate, Print, Loop*) es la consola interactiva de Scala y es una herramienta fundamental para probar expresiones y sintaxis de forma inmediata.

Característica	Descripción
Inferencia de Tipos	El REPL muestra inmediatamente el tipo deducido por el compilador, reforzando que Scala es fuertemente tipado sin requerir declaraciones

Característica	Descripción
	explícitas en cada línea.
Evaluación Eager	Las expresiones se evalúan inmediatamente y el resultado se almacena en variables temporales (<code>res0</code> , <code>res1</code> , etc.).

B. Declaración de Valores (`val` vs. `var`)

El primer concepto clave al escribir Scala es la inmutabilidad por defecto, que se establece con las palabras clave `val` y `var` :

Palabra Clave	Propósito	Mutabilidad
<code>val</code> (Value)	Immutable. La referencia no puede cambiar después de la asignación. Es la opción por defecto.	
<code>var</code> (Variable)	Mutable. La referencia puede ser reasignada. Debe evitarse en código funcional idiomático, reservándose para contextos de rendimiento o manejo de estado controlado.	

3. Ejemplos de Código Esenciales

A. Sesión REPL: Inmutabilidad e Inferencia

Una sesión de REPL es el lugar ideal para entender el funcionamiento de `val` y `var` y el tipado estático inteligente de Scala.

```

1  $ scala
2  Welcome to Scala 3.7.4 (...
3  Type in expressions for evaluation. Or try :help.
4
5  // 1. Declaración de un valor inmutable (val)
6  scala> val pi = 3.14159
7  val pi: Double = 3.14159 // El compilador infiere el tipo Double
8  // Observa que no fue necesario declarar : Double, gracias a la inferencia
   de tipos.
9
10 // 2. Intento de mutar 'val' (Produce un error en tiempo de compilación)
11 scala> pi = 4.0
12 -- Error: reassignment to val
13
14 // 3. Declaración de una variable mutable (var)
15 scala> var contador = 0
16 var contador: Int = 0

```

```

17
18 // 4. Mutación de 'var' (Válida)
19 scala> contador = contador + 1
20 contador: Int = 1

```

B. Scripting Moderno con Scala CLI (Directivas)

Para proyectos pequeños o scripts que requieren librerías externas (dependencias), Scala CLI permite usar **directivas** dentro del archivo `.scala`, eliminando la necesidad de un archivo `build.sbt` externo.

MiScript.scala

```

1 //> using scala "3.3.1"
2 // Declara la dependencia para manejar archivos (lihaoyi/os-lib)
3 //> using dep "com.lihaoyi::os-lib:0.9.1"
4
5 // Uso de una librería externa
6 object FileProcessor:
7   def main(args: Array[String]): Unit =
8     // Lógica: Muestra el contenido del directorio actual
9     val archivos = os.list(os.pwd)
10    println(s"Archivos en el directorio actual: ${archivos.size}")
11
12 // Ejecución en la terminal:
13 $ scala-cli run MiScript.scala

```

4. Comparativa con Java y Python

Aspecto	Java (Imperativo/POO)	Python (Dinámico/Imperativo)	Scala (Híbrido/Funcional)
Modelo de Mutabilidad	<code>final</code> para inmutabilidad (opcional). Mutabilidad por defecto.	Tipado dinámico. Mutabilidad simple.	<code>val</code> (inmutable) por defecto. <code>var</code> (mutable) es explícito.
Inferencia de Tipos	Limitada (<code>var</code> desde JDK 10+). Tipos explícitos requeridos para la mayoría de las declaraciones.	Tipado dinámico (resuelto en tiempo de ejecución).	Inferencia potente ; los tipos se conocen en compilación pero a menudo se omiten en la sintaxis.
Entorno de Ejecución	JVM.	Intérprete/Runtime propio.	JVM . Código compilado.

Aspecto	Java (Imperativo/POO)	Python (Dinámico/Imperativo)	Scala (Híbrido/Funcional)
Tooling Principal	Maven, Gradle (archivos XML/Groovy, verbosos).	Pip (manejo de paquetes).	sbt (proyectos grandes), Scala CLI (scripts).

Scala difiere de Python en que es **estáticamente tipado**, lo que significa que el compilador comprueba la corrección de los tipos antes de que el programa se ejecute. También se diferencia de Java en que revierte la convención de mutabilidad, prefiriendo `val` para garantizar la seguridad en la concurrencia.

5. Best Practices (*The Scala Way*)

Para empezar a programar en Scala de manera idiomática y eficiente:

1. **Prioriza `val`**: Acostúmbrate a usar `val` siempre a menos que tengas una razón explícita para usar `var` (la inmutabilidad es clave para sistemas concurrentes seguros).
2. **Adopta el REPL**: Utiliza el REPL (`scala`) constantemente para probar pequeñas expresiones, funciones de orden superior y la lógica de las librerías sin el *overhead* de la compilación completa.
3. **Tooling Flexible**: Comienza con **Scala CLI** para tus scripts y ejercicios de práctica. Solo escala a `sbt` cuando inicies un proyecto con múltiples módulos o necesites una gestión de dependencias avanzada.
4. **Entorno de Desarrollo**: Considera usar **Metals** (generalmente con VS Code) para una experiencia ligera y optimizada para Scala 3, o **IntelliJ IDEA** si trabajas en un entorno corporativo con grandes bases de código.
5. **Práctica Interactiva**: Complementa el aprendizaje en video con plataformas que proporcionan ejercicios de codificación en vivo, como **Scala Exercises** o **Tour of Scala**, lo cual ayuda a asimilar la sintaxis sin la frustración de configurar el entorno.